*Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)*

# *HASHSHIELD — Cybersecurity Hashing & File Integrity Platform*

**The domain of the Project:**

CYBER SECURITY

**Team Mentors (and their designation):**

HARI HARAN (SECURITY OPERATOR)

**Team Members:**

Mr. Manvendra Singh Raghav

**Period of the project**

6 MONTHS

November 2025 to May 2026

# Declaration

The project titled "HashShield — Cybersecurity Hashing & File Integrity Platform" has been mentored by Mr HariHaran Sir, organised by SURE Trust, from November 2025 to May 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members: Manvendra Singh Raghav

Ms. ———————— Signature

Mentor's Name Mr HariHaran
Designation—Company Name

Mentor's Name
Designation—Company

Prof. Radhakumari
Executive Director & Founder
SURE Trust

# Table of contents

# Executive Summary


- **HashShield** is a full-stack cybersecurity educational project that visually demonstrates core cryptographic concepts including SHA-256 hashing, bcrypt password security, salting, and file integrity verification. The project provides a modern dark-themed web interface with a Node.js backend, making complex security concepts interactive and accessible for learners and developers.

- The system implements six core modules: a JWT-authenticated login/signup system with bcrypt password hashing, a file upload module that generates SHA-256 fingerprints, a file integrity verifier that detects tampering, a real-time hash visualizer demonstrating the avalanche effect, a salting demonstration showing why same passwords produce different hashes, and a dashboard file registry. The stack uses Node.js, Express, bcryptjs, jsonwebtoken, multer, and the native *crypto* module.


**Project Screenshots — System Overview**

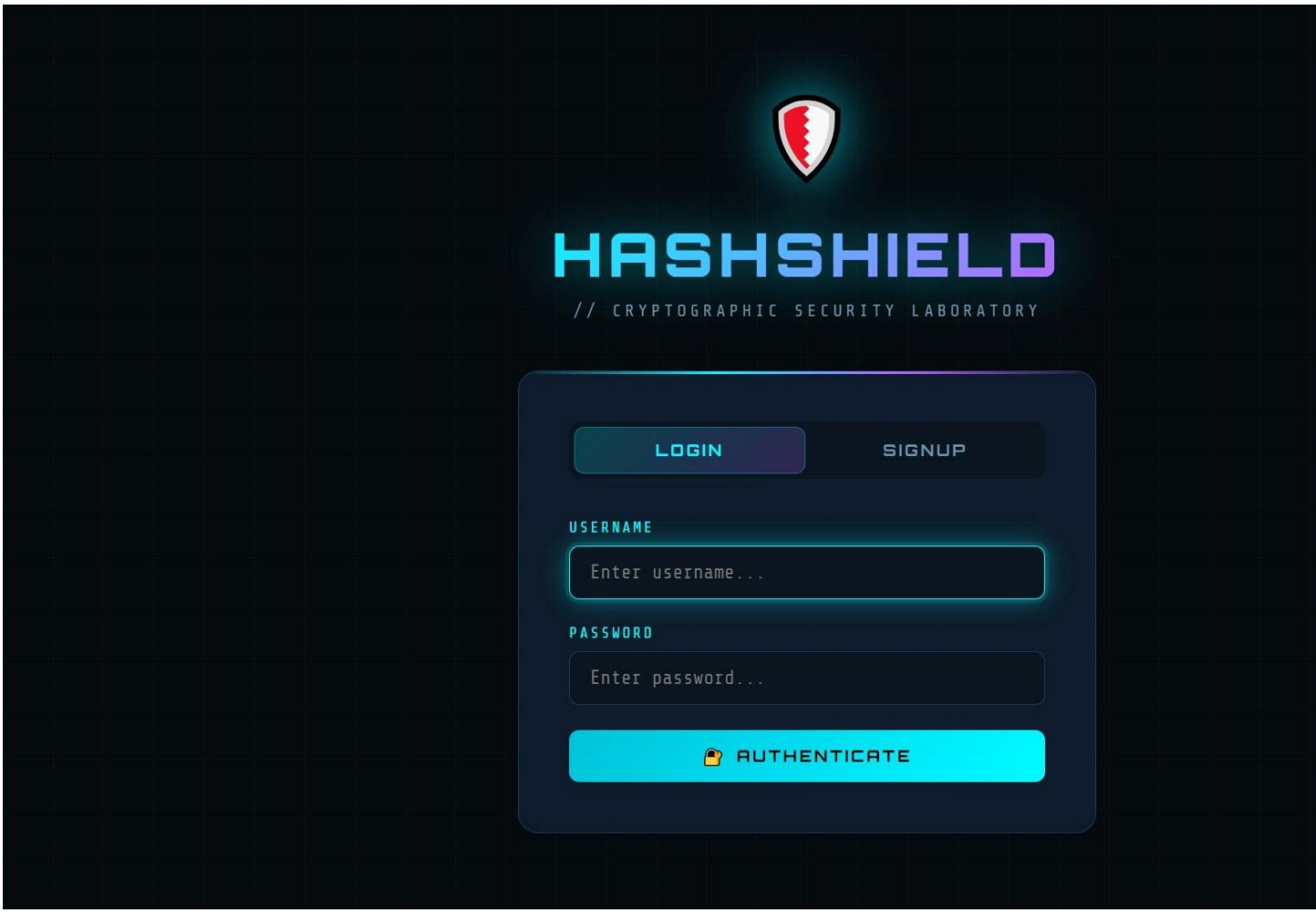


*Fig 1: HashShield Login Page — bcrypt authentication with JWT*


*Fig 2: HashShield Dashboard — File Registry & System Stats*

# Introduction

- **Background and context of the project:** In today's digital era, cybersecurity threats have grown exponentially. Yet most learners lack practical, hands-on exposure to fundamental cryptographic concepts like hashing, salting, and data integrity verification. HashShield bridges this gap by providing an interactive, visually engaging platform where users can directly observe and experiment with real cryptographic operations using SHA-256 and bcrypt.

- **Problem statement or goals of the project:** The core problem is that hashing and related concepts are often taught theoretically without practical demonstration. Students cannot visually observe how SHA-256 works, why salting matters, or how file tampering is detected. HashShield solves this by providing a real working system where every concept is demonstrated live — users upload files, see their SHA-256 hash generated instantly, re-upload modified files to detect tampering, and watch the avalanche effect in real-time.

- **Scope and limitations of the project:** The scope includes user authentication (bcrypt + JWT), file hashing (SHA-256), integrity verification, hash visualization, salting demonstration, and a file dashboard. The system uses in-memory storage for simplicity, meaning data does not persist between server restarts. It is designed as an educational cybersecurity demonstration tool.

- **Innovation component in the project:** The key innovation lies in making cryptographic concepts visually interactive. The Hash Visualizer renders SHA-256 output as 32 colored blocks in real-time. The Avalanche Effect comparator highlights exactly which hash characters change when input is modified. The Salting Demonstrator proves that identical passwords always produce different hashes using random salts. These interactive visualizations make abstract cryptographic mathematics immediately understandable.

## Project Objectives

- The primary objective of HashShield is to develop a full-stack interactive cybersecurity platform that demonstrates real-world hashing concepts in a practical, hands-on environment. The system implements SHA-256 file hashing via Node.js's native *crypto* module, bcrypt password hashing with 12 configurable salt rounds, and JWT-based session management — all integrated into a professional dark-themed cybersecurity dashboard accessible through any web browser without additional software installation.

- The expected outcome is a fully operational web application with six functional modules: (1) Secure signup/login with bcrypt and JWT; (2) File upload with SHA-256 hash generation and storage; (3) File integrity verification detecting any byte-level changes; (4) Real-time hash visualizer with avalanche effect comparison; (5) Salting demo proving same-password uniqueness; (6) Dashboard showing all files, hashes, sizes, and user info. All six modules were successfully implemented and tested.

# Methodology and Results

- **Methods/Technology used:**

  The project uses SHA-256 (via Node.js *crypto* module) for file hashing, bcryptjs (12 salt rounds) for password hashing, and JSON Web Tokens (JWT) for stateless authentication. File uploads are handled via multer with in-memory storage. The frontend uses the Web Crypto API for real-time client-side SHA-256 visualization without backend calls. RESTful API endpoints (POST /api/auth/signup, POST /api/auth/login, POST /api/file/upload, POST /api/file/verify, GET /api/file/all) handle all server-side operations.

- **Tools/Software used:**

  Backend: Node.js, Express.js, bcryptjs, jsonwebtoken, multer, cors, uuid. Frontend: HTML5, CSS3 (custom dark theme with neon effects), Vanilla JavaScript, Web Crypto API. Fonts: Google Fonts (Orbitron, Rajdhani, Share Tech Mono). Development & Testing: Visual Studio Code, Postman, browser DevTools.

- **Project Architecture:**

  The system follows a three-layer architecture: (1) Presentation Layer — single-page HTML/CSS/JS app with 6 navigable sections (Dashboard, File Upload, Verify, Hash Visualizer, Salting Demo, Concepts); (2) API Layer — Express.js REST server with JWT middleware for protected routes; (3) Security Layer — bcrypt for passwords, crypto for file hashing, JWT for sessions. File data (name, hash, size, user, timestamp) is stored in-memory as JavaScript objects.

- **Final project working screenshots with explanation:**

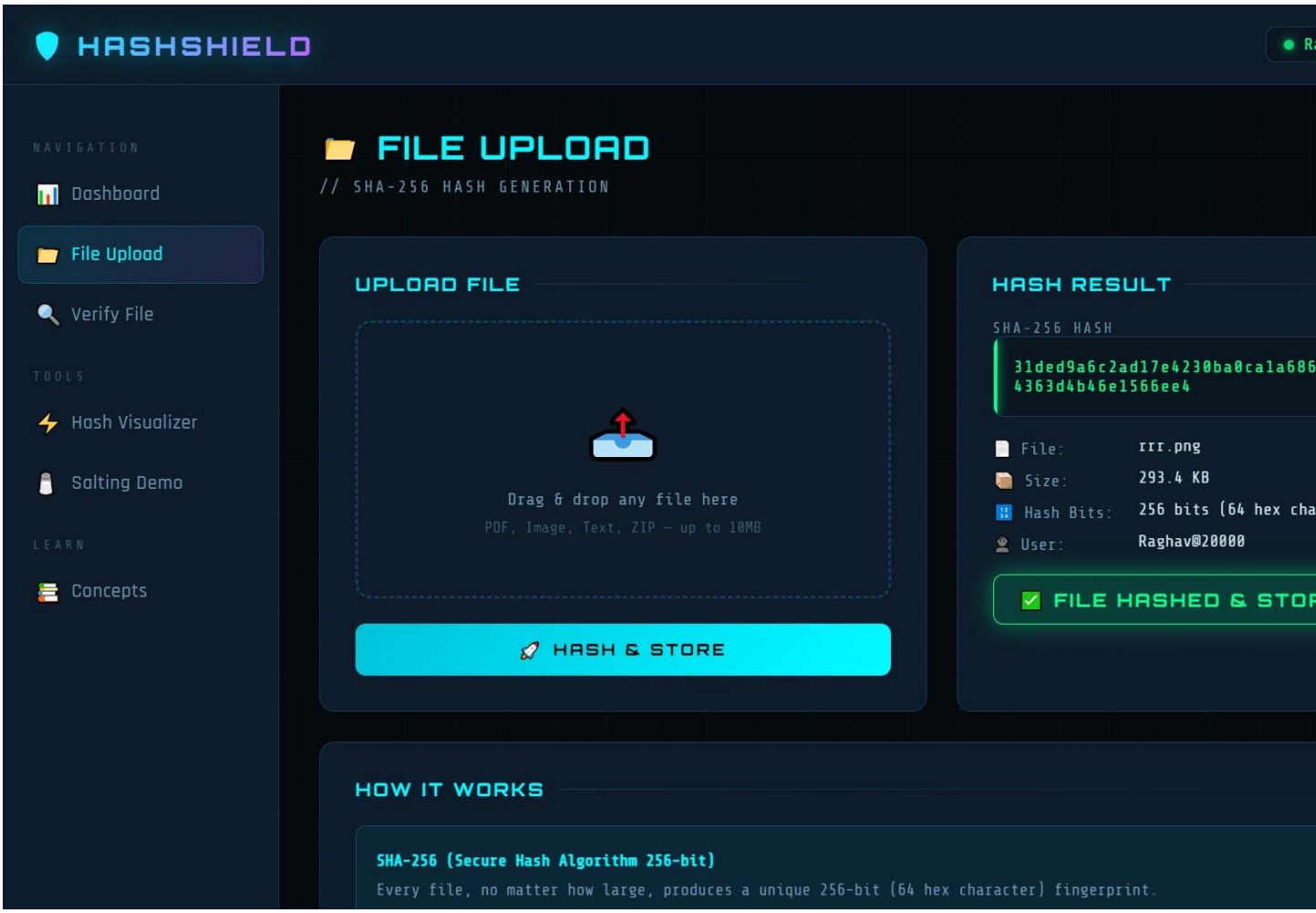


*Fig 3: File Upload — SHA-256 hash generated and stored for uploaded file*

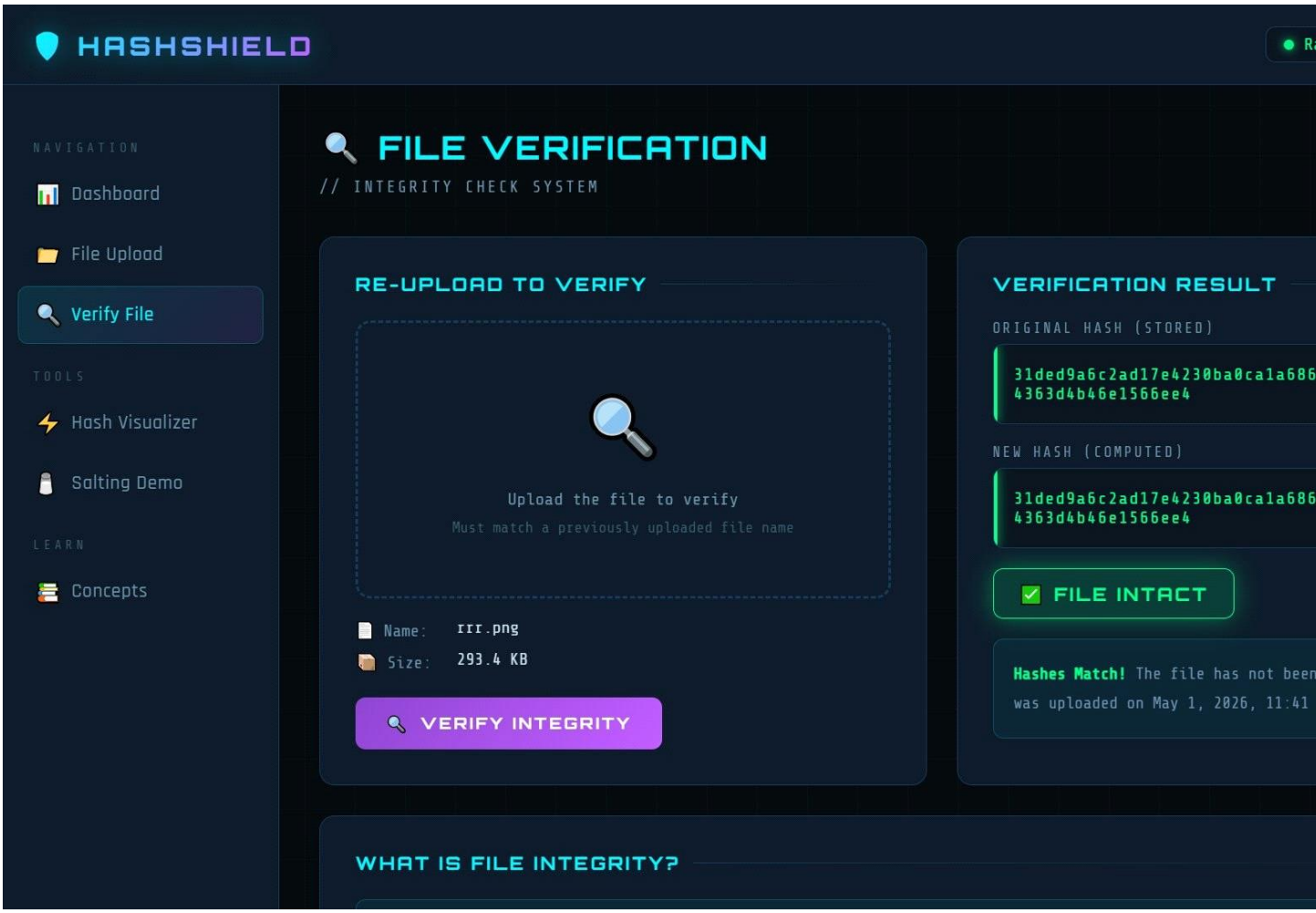


*Fig 4: File Verification — Hash comparison showing "FILE INTACT" result*

The application allows users to upload any file (PDF, image, text, ZIP) and instantly generates its SHA-256 hash, storing the file record with name, size, user, and timestamp. During verification, the same file is re-uploaded, a new hash is computed, and both hashes are compared — displaying "FILE INTACT" if matching or "FILE TAMPERED" if different. GitHub: **https://github.com/sure-trust/MANVENDRA-SINGH-RAGHAV-g14-cs**

## Social / Industry Relevance of the Project

- **Industry Relevance:** File integrity verification using cryptographic hashing is a foundational technique used across the software industry — from verifying Linux ISO downloads and npm package checksums, to blockchain transaction validation, to legal digital forensics and evidence preservation. Password hashing with bcrypt is the industry standard for secure user authentication in every major web application. HashShield demonstrates all these production-grade concepts in a single, cohesive, interactive platform.

- **Social Relevance:** Data breaches caused by plain-text password storage and tampered file transmission affect millions of users globally. By making hashing concepts visual and interactive, HashShield empowers students and developers from rural and underserved communities to understand and implement proper security practices — directly contributing to SURE Trust's mission of enabling educated rural youth with industry-relevant cybersecurity skills.

- **Educational Impact:** The avalanche effect visualizer, salting demonstrator, and real-time hash generator make abstract cryptographic mathematics tangible. A student who types "hello" and "Hello" and sees that over 60% of hash characters change immediately grasps the avalanche effect far more deeply than any textbook explanation. This experiential learning approach is the core pedagogical innovation of HashShield.

# Learning and Reflection

- **Learning (Technology & Skills)**

  Through the development of HashShield, I gained hands-on experience with cryptographic hashing algorithms, specifically SHA-256 implementation using Node.js's built-in *crypto* module. I learned how bcrypt's adaptive cost factor (salt rounds) makes password hashing intentionally slow to resist brute-force attacks. I implemented JWT-based stateless authentication and understood token signing, verification, and expiry management. Working with multer for in-memory file handling and the Web Crypto API for client-side real-time hashing deepened my full-stack development skills. Designing the dark-themed cybersecurity UI with CSS animations, neon glow effects, grid backgrounds, and responsive layout significantly improved my frontend capabilities.

- **Reflection (Overall Experience)**

  Building this project from scratch challenged me to think from both a developer and security professional's perspective. The most rewarding part was implementing the avalanche effect comparator — seeing cryptography's mathematical beauty represented visually in colored blocks. I faced challenges integrating real-time hash computation in the frontend without backend calls, which I solved using the Web Crypto API. Managing JWT middleware, file upload pipelines, and in-memory data consistency required careful architectural decisions. Overall, this project transformed my theoretical understanding of cybersecurity into practical, demonstrable skills applicable to real industry scenarios.

# Conclusion and Future Scope

- **Conclusion**

HashShield successfully achieves its objective of creating an educational, interactive cybersecurity platform that demonstrates core hashing concepts in a visually engaging manner. The project delivers a fully working system with secure user authentication (bcrypt + JWT), SHA-256 file hashing and integrity verification, a real-time hash visualizer with avalanche effect demonstration, and a salting demonstrator — all within a professional dark-themed cybersecurity dashboard. The project proves that complex cryptographic concepts can be made interactive, visual, and immediately understandable through thoughtful full-stack development.

- **Future Scope**

1. Integration with a persistent database (MongoDB/PostgreSQL) for permanent file records and multi-session data retention.
2. Cloud storage integration (AWS S3/Firebase) for actual file hosting beyond in-memory handling.
3. Addition of HMAC (Hash-based Message Authentication Code) demonstrations to show authenticated hashing.
4. Implementation of digital signature verification using RSA to complement hashing concepts.
5. Real-time multi-user collaboration features where teams can track and verify shared files.
6. Mobile-responsive PWA (Progressive Web App) version for smartphone-based learning.